Fintech Application Test Strategy

QA Portfolio

2026-05-21

Contents

1. Test Objectives

Application Context

PayFlow is a digital payments platform serving 2.5 million active users across 12 markets. The platform processes peer-to-peer transfers, merchant payments, bill splitting, and recurring scheduled payments. PayFlow integrates with 8 banking partners and 3 card networks, handling an average of 180,000 transactions per day with a peak of 450,000 during promotional events.

Primary Test Objectives

1.1 Ensure Transaction Integrity

Verify that every payment transaction maintains ACID properties from initiation through settlement. No funds should be lost, duplicated, or incorrectly routed under any condition — including partial failures, network timeouts, and concurrent operations.

1.2 Validate Regulatory Compliance

Confirm that PayFlow meets PCI-DSS Level 1 requirements, PSD2 Strong Customer Authentication (SCA) mandates, and AML/KYC verification thresholds across all supported jurisdictions.

1.3 Protect Customer Data

Verify that personally identifiable information (PII) and financial data are encrypted at rest (AES-256) and in transit (TLS 1.3), with no data leakage through logs, error messages, API responses, or third-party integrations.

1.4 Guarantee Platform Availability

Validate that PayFlow maintains 99.95% uptime SLA with graceful degradation under load, proper circuit-breaking for downstream dependencies, and recovery within defined RTO/RPO targets.

1.5 Deliver Consistent User Experience

Ensure all user-facing flows (onboarding, payment, dispute resolution) complete within performance budgets, render correctly across supported devices, and meet WCAG 2.1 AA accessibility standards.

1.6 Validate Integration Reliability

Confirm that banking partner APIs, card network gateways, and third-party services (identity verification, fraud scoring) operate correctly under normal, degraded, and failure conditions.

Success Metrics

Objective	Target KPI	Measurement Method
Transaction Integrity	0 data loss incidents per quarter	Automated reconciliation tests + production monitoring
Regulatory Compliance	100% PCI-DSS control pass rate	Quarterly compliance test suite execution
Data Protection	0 PII exposure findings per release	Security scanning + manual pen testing
Availability	<13 minutes unplanned downtime/month	Chaos engineering + synthetic monitoring
User Experience	P95 response time <800ms	Performance test suite + APM data
Integration Reliability	<0.1% partner API failure rate in tests	Contract tests + integration test results

Out of Scope

- Third-party platform testing (banking partner internal systems)
- Hardware-level testing for payment terminals
- Marketing website content testing (separate team responsibility)
- Mobile app native layer testing (handled by mobile QA team)

2. Test Scope

In-Scope Systems

Core Payment Engine

Component	Description	Test Focus
Transaction Processor	Handles payment initiation, validation, routing	Correctness, concurrency, idempotency
Ledger Service	Double-entry bookkeeping for all financial movements	Balance consistency, reconciliation
Settlement Engine	Batches and settles transactions with banking partners	Timing, retry logic, partial settlement
FX Engine	Real-time currency conversion for cross-border payments	Rate accuracy, rounding rules, margin

User-Facing Applications

Application	Platform	Test Scope
PayFlow Mobile	iOS 15+, Android 12+	E2E flows, biometric auth, push notifications
PayFlow Web	Chrome, Firefox, Safari, Edge (latest 2 versions)	Full functional testing, accessibility, responsive
Merchant Portal	Web (Chrome, Firefox)	Onboarding, transaction management, reporting
Admin Console	Internal web app	User management, compliance workflows, support tools

Integration Layer

Integration	Protocol	Test Approach
Banking Partner APIs (8)	REST/SOAP	Contract tests + integration tests
Card Networks (Visa, Mastercard, Amex)	ISO 8583 / REST	Message format validation + E2E
Identity Verification (Onfido)	REST	Mock + periodic live verification
Fraud Detection (internal ML model)	gRPC	Model accuracy + latency testing

Integration	Protocol	Test Approach
Notification Service (FCM, APNs, Email)	REST/SMTP	Delivery verification + template testing

Infrastructure

Component	Test Focus
Kubernetes cluster (EKS)	Scaling, pod recovery, resource limits
PostgreSQL (primary DB)	Data integrity, replication lag, backup/restore
Redis (caching/sessions)	Cache invalidation, failover, memory limits
Kafka (event streaming)	Message ordering, consumer lag, dead letter queue
CDN (CloudFront)	Cache headers, purge behavior, geo-routing

Out-of-Scope

- Third-party banking partner internal processing
- Physical card manufacturing and issuance
- Customer support ticketing system (Zendesk)
- Marketing analytics platform
- Corporate finance and accounting tools

Test Levels

```
                E2E / Journey Tests
          (Full user flows across services)

              Integration Tests
       (Service-to-service, DB, external APIs)

              Component Tests
         (Individual service in isolation)

                Unit Tests
        (Functions, classes, business logic)
```

Environment Coverage

Environment	Purpose	Data
Local Dev	Developer unit/integration testing	Seed data, mocked externals
CI	Automated test suite execution	Synthetic test data

Environment	Purpose	Data
Staging	Pre-release validation	Anonymized production subset
Performance	Load and stress testing	Generated high-volume data
Production	Synthetic monitoring, canary	Read-only synthetic transactions

Release Cadence

PayFlow follows a 2-week sprint cycle with the following release gates:

1. **Feature branch**: Unit tests + component tests pass
2. **PR merge to develop**: Full integration test suite passes
3. **Release candidate**: E2E suite + performance baseline + security scan
4. **Production deploy**: Canary with synthetic monitoring for 30 minutes
5. **Post-deploy**: Smoke tests + reconciliation verification

3. Risk Assessment

Risk Assessment Methodology

Each business risk is evaluated on two dimensions:

- **Likelihood** (1–5): Probability of the risk materializing in a given release cycle
  - 1 = Rare (less than once per year)
  - 2 = Unlikely (once per year)
  - 3 = Possible (once per quarter)
  - 4 = Likely (once per month)
  - 5 = Almost certain (every release)
- **Impact** (1–5): Severity of business consequences if the risk materializes
  - 1 = Negligible (cosmetic issues, no user impact)
  - 2 = Minor (workaround available, minimal user friction)
  - 3 = Moderate (partial service degradation, user complaints)
  - 4 = Major (service outage, financial loss, regulatory concern)
  - 5 = Critical (data breach, significant financial loss, regulatory action)

Risk Score = Likelihood × Impact

Coverage Priority Mapping: - Score 15–25: **Critical** — Automated tests in every pipeline, dedicated test scenarios - Score 10–14: **High** — Automated regression, regular manual exploratory testing - Score 5–9: **Medium** — Automated coverage with periodic review - Score 1–4: **Low** — Basic smoke tests, monitored in production

Risk Matrix

ID	Business Risk	Likelihood	Impact	Score	Coverage Priority
R1	Payment transaction duplication or loss during concurrent processing	3	5	15	**Critical**
R2	PCI-DSS compliance violation exposing cardholder data	2	5	10	**High**
R3	Banking partner API breaking changes causing failed settlements	4	4	16	**Critical**
R4	Race condition in balance updates leading to negative balances	3	5	15	**Critical**
R5	Authentication bypass allowing unauthorized account access	2	5	10	**High**
R6	Cross-border FX calculation errors causing financial discrepancies	3	4	12	**High**
R7	Platform unavailability during peak transaction periods	3	4	12	**High**
R8	Fraudulent transactions passing detection thresholds	4	3	12	**High**
R9	Data migration errors during schema changes corrupting user records	2	4	8	**Medium**
R10	Third-party identity verification service outage blocking onboarding	3	2	6	**Medium**

Risk Details and Mitigations

R1: Transaction Duplication/Loss (Critical)

Scenario: Under high concurrency or network partitions, the same payment could be processed twice or silently dropped, leading to incorrect balances.

Test Mitigation: - Idempotency key validation tests with concurrent duplicate requests - Chaos testing with network partitions during active transactions - Automated reconciliation tests comparing ledger entries against source events - Property-based tests for transaction state machine transitions

R2: PCI-DSS Compliance Violation (High)

Scenario: Cardholder data appears in logs, API error responses, or is stored unencrypted due to code changes.

Test Mitigation: - Automated security scanning (SAST/DAST) in every PR pipeline - Log output inspection tests verifying no PAN/CVV patterns in output - Encryption-at-rest verification tests for database fields - Annual penetration testing with interim quarterly scans

R3: Banking Partner API Breaking Changes (Critical)

Scenario: A banking partner changes their API response format or introduces new validation rules without notice, causing settlement failures.

Test Mitigation: - Consumer-driven contract tests (Pact) for each banking integration - Schema validation tests against recorded response samples - Periodic live integration verification (daily health checks) - Circuit breaker and fallback behavior tests

R4: Race Condition in Balance Updates (Critical)

Scenario: Simultaneous send and receive operations on the same account produce incorrect final balance due to read-modify-write races.

Test Mitigation: - Concurrency tests with parallel transactions on shared accounts - Database isolation level verification tests - Pessimistic locking validation under contention - Property-based tests: sum of all transactions = final balance

R5: Authentication Bypass (High)

Scenario: A flaw in session management, token validation, or biometric fallback allows unauthorized access to user accounts.

Test Mitigation: - OWASP Top 10 security test suite execution per release - Token expiration and refresh flow tests - Session fixation and replay attack tests - Privilege escalation boundary tests (user A cannot access user B data)

R6: FX Calculation Errors (High)

Scenario: Currency conversion applies stale rates, incorrect rounding, or wrong margin percentages, causing overcharging or undercharging.

Test Mitigation: - Unit tests with known rate fixtures covering all rounding scenarios - Integration tests verifying rate freshness (staleness detection) - Boundary tests at regulation-mandated maximum markup thresholds - Cross-currency transfer E2E tests with amount reconciliation

R7: Platform Unavailability During Peak (High)

Scenario: The system cannot handle peak load (3x normal traffic) during promotional events, causing timeouts and failed payments.

Test Mitigation: - Load tests simulating 3x peak traffic (450K+ transactions) - Autoscaling trigger verification tests - Graceful degradation tests (queue backpressure, rate limiting) - Chaos engineering: kill pods during load test, verify recovery

R8: Fraud Bypassing Detection (High)

Scenario: Fraudulent transaction patterns evolve and bypass the ML-based fraud detection model, resulting in unauthorized charges.

Test Mitigation: - Model accuracy regression tests against labeled fraud datasets - Rule engine boundary tests (velocity limits, geo-anomaly detection) - A/B test framework for model version comparison - Synthetic fraud pattern injection tests

R9: Data Migration Corruption (Medium)

Scenario: Database schema migrations introduce data loss, type coercion errors, or orphaned records.

Test Mitigation: - Rollback verification tests for every migration - Data integrity assertions pre/post migration on staging - Canary migration on production subset before full rollout

R10: Identity Verification Outage (Medium)

Scenario: The third-party KYC provider (Onfido) goes down, blocking new user onboarding.

Test Mitigation: - Circuit breaker tests for identity service timeout - Fallback flow tests (queued verification, manual review path) - Service health monitoring with alerting

4. Test Coverage Map

Coverage Strategy Overview

This coverage map defines the testing approach for each test type across PayFlow's application areas. Coverage targets are informed by the risk assessment (Section 3) and prioritize critical business flows.

Unit Tests

Application Area	Components Covered	Entry Criteria	Target Coverage
Transaction Processor	Payment validation, routing logic, idempotency checks, state transitions	Code compiles, dependencies mocked	90% line coverage
Ledger Service	Double-entry accounting logic, balance calculations, reconciliation algorithms	Isolated from DB, deterministic inputs	95% line coverage
FX Engine	Rate conversion, rounding rules, margin calculation, currency pair validation	Fixed rate fixtures, no external calls	95% line coverage
Fraud Detection Rules	Velocity checks, geo-anomaly scoring, pattern matching rules	Model outputs mocked	85% line coverage
User Authentication	Token generation, validation, refresh, session management	Crypto functions mocked with known keys	90% line coverage
Notification Service	Template rendering, recipient routing, retry logic	External services mocked	80% line coverage

Overall Unit Test Target: 85% aggregate line coverage across all services

Entry Criteria for Execution: - Source code compiles without errors - All external dependencies are mocked or stubbed - Test data fixtures are available and deterministic - Developer has run linting and formatting checks

Integration Tests

Application Area	Components Covered	Entry Criteria	Target Coverage
Payment Processing Pipeline	Transaction Processor → Ledger → Settlement Engine	All services deployed in CI environment, DB seeded	100% of critical paths
Banking Partner Integrations	PayFlow → Partner API Gateway (8 partners)	Contract test stubs available, network accessible	100% of API endpoints used
Database Operations	Service → PostgreSQL (CRUD, transactions, migrations)	Test database provisioned with schema applied	90% of query patterns
Event Streaming	Producer → Kafka → Consumer chains	Kafka broker running in CI, topics created	100% of event types

Application Area	Components Covered	Entry Criteria	Target Coverage
Cache Layer	Service → Redis (sessions, rate limits, cached data)	Redis instance running, flush between tests	85% of cache patterns
External Services	PayFlow → Onfido, Fraud Model, Notification	Wiremock stubs configured from recorded traffic	100% of integration points

Overall Integration Test Target: 95% of inter-service communication paths tested

Entry Criteria for Execution: - All dependent services are healthy (health check endpoints return 200) - Test database is migrated to current schema version - Message broker topics exist and are empty - Stub services are configured with expected responses - Previous test run artifacts are cleaned up

End-to-End (E2E) Tests

Application Area	User Journeys Covered	Entry Criteria	Target Coverage
User Onboarding	Registration → KYC → First deposit → Account activation	Full staging environment deployed, identity service available	100% of onboarding paths
P2P Payments	Initiate → Confirm → Process → Notify recipient	Sender and recipient accounts exist with sufficient balance	100% of payment flows
Merchant Payments	Scan QR → Enter amount → Authenticate → Confirm	Merchant account configured, terminal simulated	100% of checkout variants
Bill Splitting	Create group → Add members → Split → Collect	Multiple test accounts with relationships	90% of split scenarios
Dispute Resolution	Flag transaction → Submit evidence → Review → Resolve	Historical transactions exist, support agent account available	80% of dispute paths
Account Management	Profile update → Password change → Device management → Close account	Active account with transaction history	90% of settings flows

Overall E2E Test Target: 100% of critical user journeys, 80% of secondary flows

Entry Criteria for Execution: - Staging environment fully deployed and passing smoke tests - Test data seeded (accounts, balances, transaction history) - Third-party integrations available

(or realistic stubs for unreliable services) - Previous E2E run completed without environment-level failures - Feature flags configured to match production state

Performance Tests

Application Area	Scenarios Covered	Entry Criteria	Target/Goal
Transaction Processing	Sustained load: 3,000 TPS for 30 minutes	Performance environment provisioned at production-equivalent scale	P95 $<$ 500ms, P99 $<$ 1000ms, 0% errors
Peak Load Handling	Spike: 0 $\rightarrow$ 10,000 TPS ramp over 2 minutes	Autoscaling enabled, baseline metrics recorded	Scale within 60s, no 5xx during ramp
Database Under Load	5,000 concurrent read/write operations	Production-size dataset loaded (anonymized)	Query P95 $<$ 50ms, no deadlocks
API Gateway	50,000 concurrent connections sustained	Load balancer configured, connection pooling active	No connection refused, P95 $<$ 200ms
Settlement Batch	Process 500,000 pending settlements	Settlement queue populated, partner stubs configured	Complete within 2-hour window
Mobile App Cold Start	App launch to interactive on mid-range device	APK/IPA installed on device farm	$<$ 3 seconds to interactive

Overall Performance Target: Meet all SLA thresholds under 3x expected peak load

Entry Criteria for Execution: - Performance environment is isolated (no other workloads) - Baseline metrics captured from previous run for comparison - Monitoring and APM tools configured and recording - Test data volume matches production scale - Autoscaling policies match production configuration

Security Tests

Application Area	Test Activities	Entry Criteria	Target/Goal
Authentication & Authorization	OWASP Top 10 testing, token manipulation, privilege escalation, session hijacking	Application deployed, test accounts with various roles	Zero critical/high findings
Data Protection	PII detection in logs/responses, encryption verification, key rotation testing	Access to log aggregation, database inspection tools	Zero PII leakage findings

Application Area	Test Activities	Entry Criteria	Target/Goal
API Security	Input validation, injection attacks (SQL, NoSQL, command), rate limiting bypass	API documentation available, Burp Suite configured	Zero injection vulnerabilities
Network Security	TLS configuration, certificate pinning validation, mTLS between services	Network scanning tools configured, cert inventory available	TLS 1.3 only, no weak ciphers
Dependency Vulnerabilities	SCA scanning for known CVEs in dependencies	Package manifests accessible, vulnerability database updated	Zero critical CVEs, high CVEs patched within 7 days
Compliance Controls	PCI-DSS automated checks, data residency verification, audit log integrity	Compliance checklist mapped to automated tests	100% of automatable controls pass

Overall Security Target: Zero critical/high severity findings in production-bound releases

Entry Criteria for Execution: - Application is in a stable, testable state (no known crashes) - Security testing tools licensed and configured - Test accounts available for each role/permission level - Scope approval obtained (especially for penetration testing) - Previous security findings have been triaged (known accepted risks documented)

Coverage Gaps and Mitigation

Gap	Reason	Mitigation
Real banking partner end-to-end	Cannot test live banking in non-prod	Contract tests + periodic prod synthetic transactions
Physical device biometric testing	Device farm has limited biometric simulation	Manual testing on physical devices pre-release
Disaster recovery full rehearsal	Cost and coordination overhead	Annual DR drill, automated backup verification monthly
Regulatory changes (new markets)	Unpredictable timing	Quarterly compliance review, configurable rule engine

5. Tool Selection Rationale

Selection Criteria

Tools are evaluated against the following criteria, weighted by importance to PayFlow's testing needs:

Criterion	Weight	Description
Reliability	25%	Stability, maturity, active maintenance, community support
Integration	20%	Compatibility with existing stack (TypeScript, Java, Python, K8s)
Scalability	20%	Ability to handle growth in tests, services, and team size
Cost	15%	Licensing, infrastructure, and maintenance costs
Developer Experience	10%	Learning curve, documentation quality, debugging support
Reporting	10%	Quality of output reports, dashboards, and alerting

Category 1: Test Frameworks

Candidates Evaluated

Tool	Type	Language	License
Playwright	E2E / Browser Automation	TypeScript/JS	Apache 2.0
Cypress	E2E / Browser Automation	JavaScript	MIT
Selenium	E2E / Browser Automation	Multi-language	Apache 2.0

Evaluation

Criterion	Playwright	Cypress	Selenium
Reliability			
Integration			
Scalability			
Cost			
Developer Experience			
Reporting			

Decision: Playwright

Rationale: Playwright offers native multi-browser support (Chromium, Firefox, WebKit) without additional configuration, built-in auto-waiting that reduces flakiness, and first-class TypeScript support matching PayFlow's primary language. Its parallel execution model scales better than Cypress (which runs in-browser and has single-tab limitations). The auto-retrying assertions and trace viewer significantly reduce debugging time. Cypress was a close second for developer experience but falls short on cross-browser coverage and multi-tab/multi-origin scenarios required for PayFlow's payment redirect flows.

Category 2: CI/CD Platform

Candidates Evaluated

Tool	Type	Hosting	License
GitHub Actions	CI/CD	Cloud (GitHub-hosted)	Per-minute pricing
GitLab CI	CI/CD	Cloud or Self-hosted	Free tier + Premium
Jenkins	CI/CD	Self-hosted	MIT

Evaluation

Criterion	GitHub Actions	GitLab CI	Jenkins
Reliability			
Integration			
Scalability			
Cost			
Developer Experience			
Reporting			

Decision: GitHub Actions

Rationale: PayFlow's source code lives on GitHub, making Actions the natural choice for zero-configuration integration. The marketplace offers 15,000+ reusable actions for common tasks (Docker builds, K8s deploys, security scanning). Matrix builds enable parallel test execution across multiple OS/browser/Node combinations without custom infrastructure. The YAML-based workflow syntax is more approachable than Jenkins' Groovy DSL. GitLab CI is excellent but would require migrating the entire code hosting platform. Jenkins requires self-hosted infrastructure maintenance that diverts engineering resources.

Category 3: Performance Testing

Candidates Evaluated

Tool	Type	Protocol Support	License
k6	Load Testing	HTTP/1.1, HTTP/2, WebSocket, gRPC	AGPL-3.0
Gatling	Load Testing	HTTP/1.1, HTTP/2, WebSocket, JMS	Apache 2.0
JMeter	Load Testing	HTTP, FTP, JDBC, SOAP, LDAP	Apache 2.0

Evaluation

Criterion	k6	Gatling	JMeter
Reliability			

Criterion	k6	Gatling	JMeter
Integration			
Scalability			
Cost			
Developer Experience			
Reporting			

Decision: k6

Rationale: k6 uses JavaScript/TypeScript for test scripts, aligning with PayFlow's developer skill set and eliminating the need to learn Scala (Gatling) or navigate a GUI-based tool (JMeter). It runs as a single binary with minimal resource overhead, making it ideal for CI/CD integration. k6 supports HTTP/2 and gRPC natively — both used by PayFlow's microservices. The `k6 cloud` option provides distributed load generation when local resources are insufficient. Gatling is a strong alternative (better built-in reporting) but the Scala DSL creates a learning barrier for the team. JMeter's XML-based test plans are difficult to version control and review in PRs.

Category 4: Security Scanning

Candidates Evaluated

Tool	Type	Scan Method	License
Snyk	SAST + SCA + Container	Static analysis, dependency scanning	Commercial (free tier)
OWASP ZAP	DAST	Dynamic proxy-based scanning	Apache 2.0
SonarQube	SAST + Code Quality	Static analysis	LGPL-3.0 (Community)

Evaluation

Criterion	Snyk	OWASP ZAP	SonarQube
Reliability			
Integration			
Scalability			
Cost			
Developer Experience			
Reporting			

Decision: Snyk (SAST/SCA) + OWASP ZAP (DAST)

Rationale: PayFlow requires both static and dynamic security testing given PCI-DSS requirements. Snyk provides real-time dependency vulnerability alerts with GitHub-native integration (auto-PRs for fixes), container image scanning for our Kubernetes deployments, and IaC scanning for Terraform configs. Its developer-first approach surfaces issues early in the IDE. OWASP ZAP

complements Snyk by testing the running application for injection vulnerabilities, authentication flaws, and misconfigurations that static analysis cannot detect. SonarQube is strong for code quality but its security rules are less comprehensive than Snyk's dedicated vulnerability database for fintech compliance needs.

Category 5: API Testing

Candidates Evaluated

Tool	Type	Language	License
REST Assured	API Functional Testing	Java	Apache 2.0
Pact	Contract Testing	Multi-language	MIT
Postman/Newman	API Testing + Collection Runner	JavaScript	Commercial (free tier)

Evaluation

Criterion	REST Assured	Pact	Postman/Newman
Reliability			
Integration			
Scalability			
Cost			
Developer Experience			
Reporting			

Decision: REST Assured + Pact

Rationale: PayFlow's backend services are primarily Java-based, making REST Assured the natural fit for API functional testing with its fluent Java DSL, built-in JSON/XML parsing, and JUnit 5 integration. Pact adds consumer-driven contract testing — essential for PayFlow's 8 banking partner integrations where we cannot control the provider's release cycle. Together, they ensure both functional correctness (REST Assured) and cross-team API compatibility (Pact). Postman is excellent for exploration and ad-hoc testing but its collection-based approach doesn't integrate as cleanly into Java CI pipelines, and the paid team features create vendor lock-in.

Tool Stack Summary

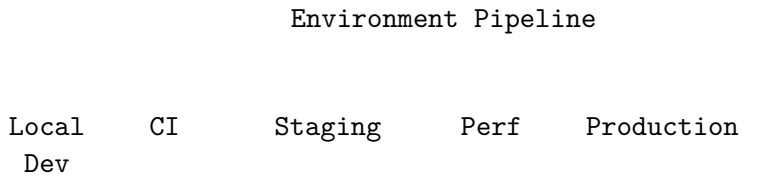
Category	Selected Tool(s)	Key Reason
Test Framework	Playwright	Multi-browser, TypeScript-native, parallel execution

Category	Selected Tool(s)	Key Reason
CI/CD	GitHub Actions	Native GitHub integration, marketplace ecosystem
Performance	k6	JavaScript scripting, lightweight, gRPC support
Security	Snyk + OWASP ZAP	SAST/SCA + DAST coverage, PCI-DSS alignment
API Testing	REST Assured + Pact	Java ecosystem fit + contract testing for integrations

6. Environment Strategy

Environment Overview

PayFlow maintains five distinct environments, each serving a specific purpose in the software delivery lifecycle. Environments are provisioned using Infrastructure as Code (Terraform) and are deployed on AWS EKS.



Environment Specifications

Local Development

Attribute	Details
Purpose	Developer unit and integration testing
Infrastructure	Docker Compose on developer machines
Services	All PayFlow services + PostgreSQL + Redis + Kafka (dockerized)
External Dependencies	Mocked via WireMock containers
Data	Seed scripts with synthetic test data
Access	Individual developer only
Refresh Cycle	On-demand (developer controls lifecycle)
Monitoring	Local logs only (stdout)

Configuration: - Services run on localhost with port mapping - Database seeded with 100 synthetic accounts and 1,000 transactions - Feature flags default to all-enabled for development - No

TLS between local services (simplified networking)

CI Environment

Attribute	Details
Purpose	Automated test suite execution on every PR and push
Infrastructure	GitHub Actions runners (ubuntu-latest) with service containers
Services	Service under test + direct dependencies (containerized)
External Dependencies	WireMock stubs, Testcontainers for databases
Data	Programmatically generated per test run (isolated)
Access	Automated pipelines only
Refresh Cycle	Ephemeral — created per workflow run, destroyed after
Monitoring	GitHub Actions logs + Allure reports as artifacts

Configuration: - Each workflow run gets a fresh environment (no state leakage) - PostgreSQL and Redis run as GitHub Actions service containers - Kafka uses Testcontainers for integration tests - Tests run in parallel across matrix builds (3 Node versions × 3 browsers) - Maximum execution time: 15 minutes per workflow

Staging

Attribute	Details
Purpose	Pre-release validation, full integration testing, UAT
Infrastructure	AWS EKS cluster (t3.medium nodes, 3-node pool)
Services	Full PayFlow platform (all microservices deployed)
External Dependencies	Sandbox environments from banking partners (where available), stubs for others
Data	Anonymized production subset (10% sample), refreshed weekly
Access	QA team, product owners, select developers
Refresh Cycle	Deployed on every merge to `develop` branch
Monitoring	DataDog APM, CloudWatch logs, Slack alerting

Configuration: - Mirrors production architecture at reduced scale (1/5th capacity) - Real TLS certificates (Let's Encrypt staging) - Feature flags controlled independently from production - Banking partner sandbox APIs integrated (Stripe test mode, partner sandboxes) - Data anonymization pipeline runs weekly from production snapshot

Performance

Attribute	Details
Purpose	Load testing, stress testing, capacity planning
Infrastructure	AWS EKS cluster (c5.2xlarge nodes, 6-node pool, production-equivalent)
Services	Full PayFlow platform at production scale
External Dependencies	High-fidelity stubs with realistic latency simulation
Data	Generated high-volume dataset (2.5M accounts, 50M transactions)
Access	QA engineers, SRE team
Refresh Cycle	On-demand, provisioned for scheduled performance test windows
Monitoring	Full observability stack (DataDog, Grafana, custom dashboards)

Configuration: - Infrastructure matches production specs (instance types, autoscaling policies) - Database pre-loaded with production-scale data volume - Network conditions simulate real-world latency to partner APIs - k6 load generators run from multiple AWS regions - Tests scheduled during off-peak hours to avoid cost spikes - Environment torn down after test window to control costs

Production

Attribute	Details
Purpose	Live service, synthetic monitoring, canary deployments
Infrastructure	AWS EKS (multi-AZ, auto-scaling 3-20 nodes per region)
Services	Full PayFlow platform (2 regions: us-east-1, eu-west-1)
External Dependencies	Live banking partner APIs, real card network connections
Data	Real customer data (encrypted, access-controlled)
Access	SRE team for deployment, read-only monitoring for QA
Refresh Cycle	Continuous deployment (canary → progressive rollout)

Attribute	Details
Monitoring	Full observability, PagerDuty alerting, SLA dashboards

Testing in Production: - Synthetic monitoring: Automated payment flows every 5 minutes using dedicated test accounts - Canary analysis: New deployments serve 5% traffic, compared against baseline for 30 minutes - Feature flag gradual rollout: New features exposed to 1% → 10% → 50% → 100% - Chaos engineering: Controlled failure injection during maintenance windows only

Data Management Strategy

Environment	Data Source	Sensitive Data Handling	Refresh Frequency
Local Dev	Seed scripts (synthetic)	No real data ever	On developer reset
CI	Generated per test run	No real data ever	Every run (ephemeral)
Staging	Production snapshot (anonymized)	PII masked/tokenized	Weekly
Performance	Generated at scale	Synthetic, statistically representative	Per test window
Production	Real customer data	Full encryption, RBAC, audit logging	N/A (live)

Anonymization Rules: - Names → randomly generated from name corpus - Emails → `user_{hash}@test.payflow.internal` - Phone numbers → `+1-555-{random 7 digits}` - Account numbers → `TEST-{random 10 digits}` - Transaction amounts → preserved (for realistic distribution) - Dates → preserved (for realistic temporal patterns)

Environment Promotion Gates

```
Local Dev → CI:        PR created, linting passes
CI → Staging:          All CI tests pass, code review approved, PR merged to develop
Staging → Performance: Staging E2E suite green, release candidate tagged
Performance → Prod:    Performance targets met, security scan clean, change approval
```

7. Entry and Exit Criteria

Purpose

Entry and exit criteria define the quality gates that must be satisfied before testing begins (entry) and before testing is considered complete (exit) at each level. These criteria prevent wasted effort on unstable builds and ensure consistent release quality.

Unit Testing

Entry Criteria

#	Criterion	Verification Method
U-E1	Code compiles without errors	Build step succeeds
U-E2	All dependencies are resolved and available	Package manager install succeeds
U-E3	Test fixtures and mock data are current with schema	Schema version check in test setup
U-E4	Developer has run code formatting and linting locally	Pre-commit hooks pass

Exit Criteria

#	Criterion	Target	Verification Method
U-X1	All unit tests pass	100% pass rate	Test runner report
U-X2	Line coverage meets threshold	85% aggregate	Coverage reporter (Istanbul/JaCoCo)
U-X3	No critical/high SAST findings	0 findings	Snyk code scan
U-X4	Mutation testing score (critical modules only)	70% killed	Stryker/PIT report

---

Integration Testing

Entry Criteria

#	Criterion	Verification Method
I-E1	Unit test exit criteria met	CI pipeline unit stage green
I-E2	All dependent services are deployed and healthy	Health check endpoints return 200
I-E3	Database schema is migrated to current version	Migration runner completes without error
I-E4	Message broker topics exist and consumers are connected	Kafka admin client verification
I-E5	External service stubs are configured	WireMock scenario verification endpoint

Exit Criteria

#	Criterion	Target	Verification Method
I-X1	All integration tests pass	100% pass rate	Test runner report
I-X2	No database deadlocks observed during test execution	0 deadlocks	PostgreSQL pg_stat_activity monitoring
I-X3	No unhandled message broker errors	0 dead-letter messages	Kafka DLQ consumer count
I-X4	API response times within threshold	P95 $<$ 500ms	Test execution metrics
I-X5	Contract tests pass for all consumer-provider pairs	100% verified	Pact verification report

---

End-to-End Testing

Entry Criteria

#	Criterion	Verification Method
E-E1	Integration test exit criteria met	CI pipeline integration stage green
E-E2	Staging environment fully deployed (all services)	Deployment manifest verified, health checks pass
E-E3	Test data seeded (accounts, balances, history)	Seed script execution report
E-E4	Third-party sandboxes accessible	Connectivity check to partner APIs
E-E5	Feature flags configured to match target release state	Feature flag audit report
E-E6	No P1/P2 bugs open from previous E2E cycle	Bug tracker query

Exit Criteria

#	Criterion	Target	Verification Method
E-X1	Critical user journeys pass	100% pass rate	Allure report — critical tag filter
E-X2	Overall E2E pass rate	95%	Allure report summary
E-X3	No new P1 (critical) defects found	0 unresolved P1s	Bug tracker
E-X4	P2 (high) defects triaged and accepted/deferred	All P2s have disposition	Bug tracker audit

#	Criterion	Target	Verification Method
E-X5	All flaky tests identified and quarantined	Flaky tests labeled, not blocking	Test report flaky flag
E-X6	Accessibility scan clean	0 WCAG 2.1 AA violations	axe-core report

---

Performance Testing

Entry Criteria

#	Criterion	Verification Method
P-E1	E2E exit criteria met (functional correctness confirmed)	E2E pipeline stage green
P-E2	Performance environment provisioned at production scale	Infrastructure diff report
P-E3	Test data loaded at production volume	Data generation script report
P-E4	Baseline metrics from previous release available	Metrics archive accessible
P-E5	Monitoring and APM tools active and recording	Dashboard connectivity verified
P-E6	No other workloads running on performance environment	Resource utilization $< 10\%$

Exit Criteria

#	Criterion	Target	Verification Method
P-X1	Transaction processing throughput	3,000 TPS sustained for 30 min	k6 summary report
P-X2	Response time under load	P95 $<$ 500ms, P99 $<$ 1000ms	k6 percentile report
P-X3	Error rate under load	$< 0.1\%$	k6 error metrics
P-X4	No memory leaks detected	Heap growth $<$ 5% over test duration	APM memory profiling
P-X5	Autoscaling triggers correctly	Pods scale within 60s of threshold	K8s HPA event log
P-X6	No performance regression vs. previous release	$< 10\%$ degradation on key metrics	Baseline comparison report

Security Testing

Entry Criteria

#	Criterion	Verification Method
S-E1	Application is in a stable, testable state	Latest staging deployment healthy
S-E2	Security testing tools configured and updated	Tool version and rule update check
S-E3	Test accounts for all roles/permission levels available	Account provisioning script
S-E4	Scope document approved by security team	Signed scope document
S-E5	Previous security findings have been triaged	Finding tracker — all items have status

Exit Criteria

#	Criterion	Target	Verification Method
S-X1	SAST findings (critical/high)	0 unresolved	Snyk dashboard
S-X2	DAST findings (critical/high)	0 unresolved	ZAP scan report
S-X3	Dependency vulnerabilities (critical)	0 critical CVEs	Snyk SCA report
S-X4	PCI-DSS automated controls	100% pass	Compliance test suite report
S-X5	Penetration test findings (critical/high)	0 unresolved	Pen test report
S-X6	All findings documented with remediation plan	100% triaged	Security tracker

Release Criteria (Production Deployment Gate)

Final Go/No-Go Checklist

#	Criterion	Owner	Sign-off Required
R1	All test level exit criteria met	QA Lead	
R2	No open P1 or unaccepted P2 defects	QA Lead	

#	Criterion	Owner	Sign-off Required
R3	Security scan clean (zero critical/high)	Security Lead	
R4	Performance targets met	Performance Engineer	
R5	Rollback plan documented and tested	SRE Lead	
R6	Release notes reviewed	Product Manager	
R7	Change advisory board approval (if applicable)	Change Manager	
R8	Monitoring dashboards and alerts configured for new features	SRE Lead	

Suspension Criteria

Testing will be suspended if any of the following occur:

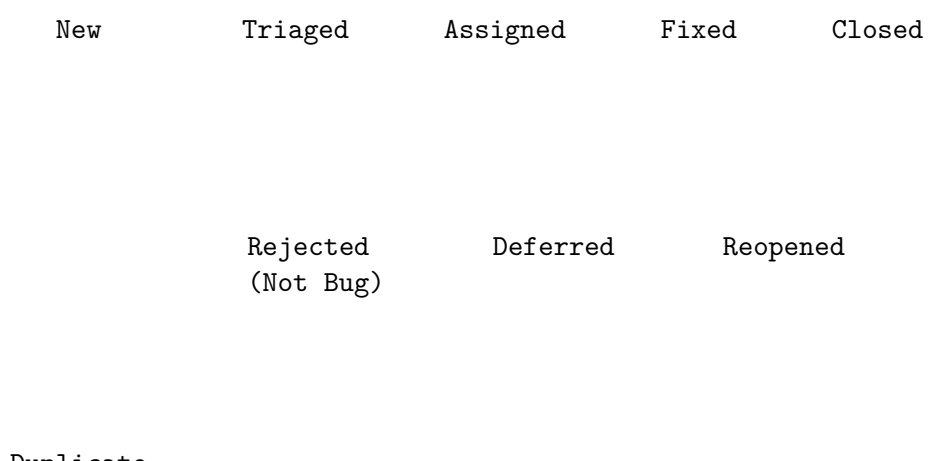
1. **Environment instability**: More than 3 environment-caused test failures in a 1-hour window
2. **Critical defect found**: P1 defect that blocks further meaningful test execution
3. **Data corruption**: Test data integrity compromised requiring re-seeding
4. **Security incident**: Active security concern requiring immediate investigation
5. **External dependency outage**: Third-party service unavailable with no stub fallback

Resumption Criteria

Testing resumes when: - Root cause of suspension is identified and resolved - Environment stability is confirmed (30-minute clean run) - Test data integrity is verified - QA Lead approves resumption

8. Defect Management

Defect Lifecycle



Severity Classification

Severity	Definition	Examples	SLA (Response)	SLA (Resolution)
P1 — Critical	System unusable, data loss/corruption, security breach, financial impact	Payment processing failure, authentication bypass, data breach	30 minutes	4 hours
P2 — High	Major feature broken, no workaround, significant user impact	Cannot complete payment flow, incorrect balance display, settlement failure	2 hours	24 hours
P3 — Medium	Feature partially broken, workaround exists, moderate user impact	Notification delay, report export error with manual alternative, slow page load	8 hours	5 business days
P4 — Low	Cosmetic issue, minor inconvenience, edge case	Typo in confirmation message, slight layout shift, tooltip misalignment	24 hours	Next sprint

Defect Report Template

Every defect report must include the following fields:

```
## Bug Report

**Title:** [Concise description of the issue]

**Severity:** P1 / P2 / P3 / P4
**Priority:** Critical / High / Medium / Low
**Component:** [Service or module affected]
**Environment:** [Where discovered: CI, Staging, Performance, Production]
```

```
**Build/Version:** [Release tag or commit SHA]
**Reporter:** [Name]
**Date Found:** [YYYY-MM-DD]

### Description
[2-3 sentence summary of the issue and its user impact]

### Steps to Reproduce
1. [Precondition]
2. [Action]
3. [Action]
4. [Observed result]

### Expected Result
[What should happen]

### Actual Result
[What actually happened]

### Evidence
- Screenshots/Screen recordings: [attachment]
- Logs: [relevant log snippet or link]
- Network trace: [HAR file or API response]

### Impact Assessment
- Users affected: [count or percentage]
- Financial impact: [estimated if applicable]
- Workaround available: Yes/No - [description if yes]

### Technical Notes
[Root cause hypothesis, related code areas, suggested fix approach]
```

Triage Process

Daily Triage Meeting (15 minutes)

Participants: QA Lead, Dev Lead, Product Owner **Frequency:** Daily at 09:30 during active testing phases; 3x/week otherwise **Agenda:** 1. Review new defects (5 min) 2. Assign severity and priority (5 min) 3. Assign owner and target sprint/release (5 min)

Triage Decision Framework

```
Is it reproducible?
  No → Request more info, move to "Needs Info"
  Yes
      Is it a regression?
          Yes → Automatic P2+ severity, assign to sprint
          No
```

```
Assess business impact
    Financial/Security/Data risk → P1 or P2
    Feature broken, no workaround → P2
    Feature broken, workaround exists → P3
    Cosmetic/Edge case → P4
```

Defect Metrics and Reporting

Key Metrics Tracked

Metric	Calculation	Target	Review Cadence
Defect Discovery Rate	New defects / week	Trending down sprint-over-sprint	Weekly
Defect Escape Rate	Production defects / total defects found	$< 5\%$	Per release
Mean Time to Detect (MTTD)	Time from introduction to discovery	$<$ 1 sprint	Per release
Mean Time to Resolution (MTTR)	Time from report to verified fix	P1: $<$ 4h, P2: $<$ 24h	Weekly
Defect Reopen Rate	Reopened defects / total fixed defects	$< 10\%$	Per sprint
Test Effectiveness	Defects found in test / total defects	$> 90\%$	Per release
Defect Density	Defects per KLOC (thousand lines of code)	$<$ 5 per KLOC	Per release

Sprint-Level Dashboard

The defect dashboard displays: - Open defect count by severity (bar chart) - Defect aging (time in each state) - Discovery phase breakdown (unit, integration, E2E, production) - Top defect-producing components (Pareto chart) - SLA compliance percentage

Release-Level Report

Each release includes a quality summary with: - Total defects found during testing cycle - Defects by severity breakdown - Defect escape count (found post-release) - Comparison against previous 3 releases (trend) - Root cause categories (code logic, integration, configuration, environment)

Defect Prevention

Root Cause Analysis (RCA)

For every P1 and P2 defect, a mini-RCA is conducted:

Step	Action	Output
1	Identify root cause	Technical cause documented
2	Determine injection phase	When was the defect introduced (design, code, config)
3	Determine escape reason	Why wasn't it caught earlier
4	Define prevention action	Specific improvement (new test, rule, process change)
5	Implement prevention	PR with test/fix, process update committed

Continuous Improvement Actions

- **Test gap analysis**: After every escaped defect, add a test that would have caught it
- **Flaky test management**: Quarantine flaky tests immediately, fix within 2 sprints
- **Code review checklist**: Update based on common defect root causes
- **Static analysis rules**: Add custom rules for repeated defect patterns
- **Knowledge sharing**: Monthly "Bug of the Month" presentation highlighting learnings

Tools

Tool	Purpose	Integration
GitHub Issues	Defect tracking and lifecycle management	Native to repository
GitHub Projects	Sprint board and workflow automation	Linked to Issues
Allure	Test execution reporting with failure details	CI pipeline artifact
DataDog	Production defect detection and alerting	APM + log monitoring
Slack	Real-time P1/P2 notifications	GitHub + DataDog webhooks

Escalation Path

```
P4/P3: QA Engineer → Dev Team (normal sprint flow)
P2:    QA Lead → Dev Lead → resolved within 24h
P1:    QA Lead → Engineering Manager → CTO (if > 2h without fix)
        → Incident commander assigned
        → War room opened
        → Stakeholder communication every 30 minutes
```